

Propuesta de Solución: Ejercicio 2

Este documento expone la propuesta de mejora para el código del backend, centrada en la optimización del código repetitivo (boilerplate) mediante el uso de anotaciones de Lombok en los modelos JPA.

Hallazgo: Código Boilerplate Redundante en Modelos

Al analizar los modelos JPA en el backend (`Jugador.java`, `Equipo.java`, `CartaFUT.java` y `Alineacion.java`), se constata que todos contienen constructores vacíos, constructores con todos los campos, getters y setters implementados manualmente por escrito. Dado que la dependencia de **Lombok** ya está añadida en el archivo `pom.xml` del proyecto, esta redundancia puede simplificarse de forma óptima.

Solución Propuesta: Constructores Automáticos con Lombok

Podemos optimizar la declaración de las clases de modelo utilizando anotaciones de la librería Lombok en la cabecera de la clase:

- **@NoArgsConstructor**: Genera automáticamente el constructor sin argumentos (obligatorio para la persistencia de JPA / Hibernate).
- **@AllArgsConstructor**: Genera de forma automática un constructor parametrizado que acepta un argumento por cada atributo definido en la clase.
- **@Getter / @Setter**: Autogeneran los métodos de acceso y asignación en tiempo de compilación.

Comparativa de Código (Caso de Estudio:

Alineacion.java)

Antes (Estructura Manual Extensa - 69 líneas):

```
package com.futmanager.model;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
public class Alineacion {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String formacion;

    @OneToMany(mappedBy = "alineacion", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<AlineacionPosicion> posiciones = new ArrayList<>();

    // Constructor vacío manual
    public Alineacion() {}

    // Constructor con argumentos manual
    public Alineacion(Long id, String nombre, String formacion) {
        this.id = id;
        this.nombre = nombre;
        this.formacion = formacion;
    }

    // Getters y Setters manuales (Ocupan gran espacio)...
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}
```

Después (Implementación Limpia con Lombok - 32 líneas):

```
package com.futmanager.model;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import java.util.ArrayList;
import java.util.List;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Alineacion {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String formacion;

    @OneToMany(mappedBy = "alineacion", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<AlineacionPosicion> posiciones = new ArrayList<>();
}
```



Beneficios de la Mejora

- **Legibilidad del código:** Permite a los desarrolladores enfocar la lectura directamente sobre los atributos y relaciones JPA sin ruido visual de boilerplate.
- **Facilidad de mantenimiento:** Al agregar o modificar campos, Lombok regenera los constructores y accesores automáticamente en la compilación.

- **Reducción del tamaño físico del archivo:** Reducción superior al 50% en las líneas de código de las clases.